
ESTÁNDAR DE CIFRADO AVANZADO (AES): IMPLEMENTACIÓN Y ANÁLISIS ADVANCED

ENCRYPTION STANDARD (AES): IMPLEMENTATION AND ANALYSIS

Integrantes:

Nicolás Agustín Marco

Ingeniería en Informática, Universidad Nacional de La Matanza

`nmarco@alumno.unlam.edu.ar`

Iván Fabrizio Abaca

Ingeniería en Informática, Universidad Nacional de La Matanza

`iabaca@alumno.unlam.edu.ar`

Florencia Berenice Licarzi

Ingeniería en Informática, Universidad Nacional de La Matanza

`flicarzi@alumno.unlam.edu.ar`

Luca Sebastian Gonzalez

Ingeniería en Informática, Universidad Nacional de La Matanza

`lucasebgonzalez@alumno.unlam.edu.ar`

Augusto Martin Coronati

Ingeniería en Informática, Universidad Nacional de La Matanza

`acoronati@alumno.unlam.edu.ar`

Franco Caccavari

Ingeniería en Informática, Universidad Nacional de La Matanza

`fcaccavari064@alumno.unlam.edu.ar`

Lautaro Leonardo Nicolas Schiaffino

Ingeniería en Informática, Universidad Nacional de La Matanza

`laschiaffino@alumno.unlam.edu.ar`

Federico Gabriel Castro

Ingeniería en Informática, Universidad Nacional de La Matanza

`federiccastro@alumno.unlam.edu.ar`

Referente Institucional:

Carla, BARROS

Mónica, LORDI

Jorge Alejandro, SILVESTRI

Alesio Esteban Abell, SINOPOLI

Martín Hernan, ZEBALLOS

Universidad Nacional de la Matanza

Resumen:

Este trabajo presenta la implementación desde cero del Estándar de Cifrado Avanzado (AES-128) en lenguaje Python, sin recurrir a librerías criptográficas externas. La implementación abarca la primitiva completa definida en FIPS-197 —incluyendo la aritmética en el cuerpo finito $GF(2^8)$, las cuatro transformaciones de ronda y el algoritmo de expansión de claves— así como los modos de operación ECB (Electronic Codebook) y CBC (Cipher Block Chaining) con relleno PKCS#7. La corrección fue verificada mediante vectores de prueba oficiales del NIST (FIPS-197 y SP 800-38A), obteniendo resultados coincidentes en los 18 casos de prueba ejecutados. Como caso de estudio, se cifró una imagen sintética en formato BMP con ambos modos, demostrando visualmente la vulnerabilidad de ECB ante datos con patrones repetitivos y la resistencia de CBC. El trabajo permite comprender en profundidad los principios de confusión y difusión formulados por Shannon y su aplicación concreta en uno de los algoritmos criptográficos más utilizados del mundo.

Abstract:

This paper presents a from-scratch implementation of the Advanced Encryption Standard (AES-128) in Python, without relying on external cryptographic libraries. The implementation covers the complete primitive defined in FIPS-197 — including arithmetic in the finite field $GF(2^8)$, the four round transformations, and the key expansion algorithm — as well as the ECB (Electronic Codebook) and CBC (Cipher Block Chaining) modes of operation with PKCS#7 padding. Correctness was verified against official NIST test vectors (FIPS-197 and SP 800-38A), producing matching results across all 18 test cases. As a case study, a synthetic BMP image was encrypted using both modes, visually demonstrating ECB's vulnerability to data with repetitive patterns and CBC's resistance. The work provides an in-depth understanding of Shannon's confusion and diffusion principles and their concrete application in one of the most widely used cryptographic algorithms in the world.

Palabras Clave: *AES, criptografía simétrica, cifrado de bloque*

Key Words: *AES, symmetric cryptography, block cipher*

I. CONTEXTO

Este trabajo se realizó en el marco de la materia **Matemática Aplicada**, dentro del área de Criptografía y Seguridad Informática, a cargo del docente Martín Hernán Zeballos. El objetivo fue implementar desde cero una primitiva criptológica simétrica ampliamente utilizada (AES), comprendiendo tanto su funcionamiento interno como su rol en sistemas reales de cifrado/descifrado.

II. INTRODUCCIÓN

A. El Problema

El cifrado simétrico resuelve un problema central de la criptografía: cómo transformar información legible (texto plano) en información ilegible (texto cifrado) de forma que solo quien posea una clave secreta compartida pueda revertir el proceso. A diferencia de la criptografía asimétrica (como RSA o Diffie-Hellman), en el cifrado simétrico la misma clave se usa para cifrar y descifrar, lo que lo hace mucho más eficiente computacionalmente — por eso es el método elegido cuando hay que cifrar grandes volúmenes de datos (archivos, tráfico de red, discos completos).

Dentro del cifrado simétrico existen dos grandes familias: cifradores de flujo (procesan bit a bit o byte a byte, como ChaCha20) y cifradores de bloque (procesan bloques de tamaño fijo, como AES). AES es un cifrador de bloque: opera sobre bloques de 128 bits (16 bytes) sin importar el tamaño de la clave.

B. Antecedentes y Origen

Hasta fines de los años 90, el estándar de cifrado simétrico era **DES** (Data Encryption Standard), adoptado por el NIST (National Institute of Standards and Technology, EE.UU.) en 1977 [1]. DES usaba una clave de 56 bits — un tamaño que en 1977 se consideraba seguro, pero que para los años 90 ya era vulnerable a ataques de fuerza bruta con hardware especializado. En 1998, la máquina “Deep Crack” del EFF rompió una clave DES en menos de 3 días [2], dejando claro que el algoritmo estaba obsoleto.

Como respuesta, en 1997 el NIST convocó a un **concurso público internacional** para definir un nuevo estándar: el AES (Advanced Encryption Standard) [3]. Se presentaron 15 algoritmos candidatos de equipos de todo el mundo, evaluados en rondas sucesivas según criterios de seguridad, eficiencia (en software y hardware) y flexibilidad. En la ronda final quedaron cinco finalistas: MARS, RC6, Rijndael, Serpent y Twofish.

En octubre de 2000, el NIST anunció como ganador a **Rijndael**, diseñado por los criptógrafos belgas **Joan Daemen y Vincent Rijmen** [4]. En noviembre de 2001 se publicó oficialmente como el estándar **FIPS-197** [5], y desde entonces AES es el cifrador simétrico de bloque más utilizado en el mundo.

Un dato relevante del proceso: a diferencia de algoritmos diseñados en secreto por agencias gubernamentales (como ocurrió parcialmente con DES), el diseño de Rijndael y todo el proceso de evaluación fueron **públicos y abiertos al escrutinio de la comunidad criptográfica internacional** — un principio conocido como “seguridad por diseño abierto”, que contrasta con la “seguridad por oscuridad” y que hoy se considera una buena práctica fundamental.

C. Propiedades y Estructura

AES es una red de sustitución-permutación (SPN) que opera sobre un bloque de 128 bits organizado como una matriz de 4x4 bytes (el “estado”). Admite tres tamaños de clave: 128, 192 o 256 bits, lo que determina el número de rondas de transformación: 10, 12 o 14 respectivamente [5].

Cada ronda (salvo la última, que omite un paso) aplica cuatro transformaciones:

- SubBytes: sustitución no lineal byte a byte mediante una tabla fija (la “S-box”), que aporta confusión — oculta la relación matemática entre la clave y el texto cifrado.
- ShiftRows: desplazamiento cíclico de las filas del estado, que aporta difusión — esparce la influencia de cada byte de entrada sobre múltiples bytes de salida.
- MixColumns: mezcla las columnas del estado mediante multiplicación de polinomios en el cuerpo finito $GF(2^8)$, reforzando la difusión entre bytes de una misma columna.
- AddRoundKey: combina el estado con una “clave de ronda” derivada de la clave original mediante el algoritmo de expansión de claves (key schedule), usando XOR.

Estos dos conceptos — confusión (SubBytes) y difusión (ShiftRows + MixColumns) — fueron formulados originalmente por Claude Shannon en 1949 [6] como los dos pilares de un cifrado seguro, y AES es uno de los ejemplos más estudiados de su aplicación práctica.

D. Ámbito de Aplicabilidad

AES es, hoy en día, el cifrador simétrico más extendido del mundo. Algunos ejemplos de su uso:

- TLS/HTTPS: la mayoría de las conexiones cifradas a sitios web usan AES (en modo GCM) como cifrador de la sesión, una vez establecida la clave compartida mediante Diffie-Hellman o similar.
- Almacenamiento: BitLocker (Windows), FileVault (macOS) y LUKS (Linux) usan AES para cifrar discos completos.
- Redes inalámbricas: WPA2 y WPA3 usan AES (en modo CCMP) para proteger el tráfico WiFi.
- VPNs: OpenVPN, WireGuard y IPsec utilizan AES como cifrador de datos.
- Uso gubernamental: la NSA aprobó AES-128 para información clasificada hasta “SECRET” y AES-192/256 para “TOP SECRET” [7], lo cual es un indicador fuerte de la confianza depositada en el algoritmo más de dos décadas después de su publicación.

A más de 20 años de su estandarización, y pese a un volumen enorme de criptoanálisis académico, no se conoce ningún ataque práctico que rompa AES en sus configuraciones estándar — los únicos ataques exitosos conocidos (related-key attacks) requieren condiciones que no se dan en implementaciones reales [8].

III. MÉTODOS

A. Diseño

El sistema implementado consta de tres componentes:

1. Núcleo AES-128: una implementación desde cero (sin librerías criptográficas externas) de las primitivas definidas en FIPS-197, capaz de cifrar y descifrar un bloque individual de 128 bits.
2. Modos de operación: sobre el núcleo, se implementaron los modos ECB (Electronic Codebook) y CBC (Cipher Block Chaining), con relleno (padding) PKCS#7 para mensajes cuya longitud no es múltiplo de 16 bytes.
3. Herramienta de cifrado/descifrado: una interfaz de línea de comandos que permite cifrar y descifrar archivos (texto e imágenes) eligiendo clave y modo de operación.

Como caso de estudio didáctico, se utilizó el sistema para cifrar una imagen en formato BMP en modo ECB y en modo CBC, comparando los resultados — este experimento es un ejemplo clásico para ilustrar por qué ECB no es un modo seguro para datos con patrones repetitivos.

B. Implementación Matemática

Estructura del estado: AES-128 opera sobre un bloque de 128 bits organizado como una matriz de 4x4 bytes (el “estado”), y aplica 10 rondas de transformación.

SubBytes: cada byte del estado se sustituye por el valor correspondiente en la S-box, una tabla fija de 256 valores definida por FIPS-197 a partir de la inversa multiplicativa en $GF(2^8)$ compuesta con una transformación afín. Para el descifrado se usa la S-box inversa.

ShiftRows: las filas de la matriz de estado se rotan cíclicamente hacia la izquierda (fila i se rota i posiciones). En el descifrado, la rotación es hacia la derecha (InvShiftRows).

MixColumns: cada columna del estado se multiplica por una matriz fija, donde las operaciones de suma y multiplicación se realizan en el cuerpo finito $GF(2^8)$ módulo el polinomio irreducible $x^8+x^4+x^3+x+1$. La suma en $GF(2^8)$ es un XOR; la multiplicación se implementó mediante el algoritmo de “multiplicación rusa de campesinos” adaptado a este cuerpo. InvMixColumns usa la matriz inversa.

AddRoundKey: el estado se combina con la clave de ronda correspondiente mediante XOR byte a byte.

Key Expansion: a partir de la clave de 128 bits (16 bytes / 4 palabras de 32 bits) se generan 11 claves de ronda (44 palabras en total) mediante un proceso iterativo que combina rotación de palabras (RotWord), sustitución con la S-box (SubWord) y constantes de ronda (Rcon).

Padding: para los modos ECB y CBC, los mensajes se completan al múltiplo de 16 bytes más cercano usando PKCS#7 (se añaden N bytes de valor N).

C. Estructura del Código

El proyecto se organizó en los siguientes módulos (Python):

- aes/core.py: constantes (S-box, S-box inversa, Rcon), key expansion, y las funciones de cifrado/descifrado de un bloque de 16 bytes.
- aes/modes.py: implementación de ECB y CBC sobre el núcleo, incluyendo padding/unpadding PKCS#7.

- cli.py: interfaz de línea de comandos para cifrar/descifrar archivos.
- tests/tests.py: validación del núcleo contra los vectores de prueba oficiales de FIPS-197 (Apéndice B y C).
- demo/demo_ecb_cbc.py: script que cifra una imagen BMP en ambos modos y genera las imágenes resultantes para comparación visual.

D. Herramientas Utilizadas

- Lenguaje Python 3, sin librerías criptográficas externas para el núcleo (operaciones de bytes, listas y aritmética modular puras).
- Librería estándar para manejo de archivos e imágenes BMP (formato simple que permite manipular los datos de píxeles directamente).
- Vectores de prueba oficiales del NIST (FIPS-197) para validación de corrección.

IV. RESULTADOS Y OBJETIVOS

A. Validación del núcleo AES-128 contra vectores del NIST

La corrección de la implementación fue verificada mediante 18 casos de prueba organizados en cinco grupos, ejecutados contra los vectores de prueba oficiales del NIST:

FIPS-197 Apéndice B: vector canónico de bloque único con clave 2b7e151628aed2a6abf7158809cf4f3c y plaintext 3243f6a8885a308d313198a2e0370734. El ciphertext obtenido (3925841d02dc09fbdc118597196a0b32) coincidió exactamente con el publicado en el estándar. El descifrado recuperó el plaintext original.

FIPS-197 Apéndice C.1: segundo vector de bloque único con clave 000102030405060708090a0b0c0d0e0f y plaintext 00112233445566778899aabbccddeeff. Ciphertext obtenido: 69c4e0d86a7b0430d8cdb78070b4c55a, coincidente con FIPS-197. Descifrado correcto.

NIST SP 800-38A – ECB-AES128: cuatro bloques de 16 bytes cifrados con la misma clave del Apéndice B. Se verificó cada bloque de forma individual y la concatenación de los cuatro. Todos los ciphertexts obtenidos coincidieron con los vectores de SP 800-38A, y el descifrado recuperó el plaintext en todos los casos.

NIST SP 800-38A – CBC-AES128: mismos cuatro plaintexts cifrados en modo CBC con IV 000102030405060708090a0b0c0d0e0f. Los ciphertexts obtenidos coincidieron con los vectores del estándar; el descifrado fue correcto.

Prueba de ida y vuelta: el mensaje arbitrario "Hola AES!" Este mensaje no es múltiplo de 16 bytes." (50 bytes, no múltiplo de 16) fue cifrado con padding PKCS#7 en modo ECB y en modo CBC; en ambos casos el descifrado recuperó exactamente el mensaje original, verificando la correcta implementación del padding.

Los 18 tests pasaron sin errores. La coincidencia exacta con los vectores del NIST — en particular con los del Apéndice B de FIPS-197, que publican el estado interno del cifrador ronda por ronda — confirma que cada transformación (SubBytes, ShiftRows, MixColumns, AddRoundKey y Key Expansion) fue implementada correctamente.

B. Experimento visual: ECB vs. CBC sobre imagen BMP

Se generó una imagen sintética de 128×128 píxeles en formato BMP sin compresión, compuesta por una grilla de 4×4 bloques de 32×32 píxeles, cada uno con un color uniforme distinto. Los datos de píxeles totalizan 49.152 bytes ($128 \times 128 \times 3$ bytes BGR), que corresponden a exactamente 3.072 bloques AES de 16 bytes. El archivo completo —incluyendo el encabezado BMP de 54 bytes— ocupa 49.206 bytes.

La clave utilizada fue 2b7e151628aed2a6abf7158809cf4f3c y el IV para CBC 000102030405060708090a0b0c0d0e0f.

Cifrado en modo ECB: dado que los datos de píxeles son múltiplo exacto de 16 bytes, no se agrega padding. Dentro de cada bloque de 32×32 píxeles de color uniforme, los bytes de los tres canales (B, G, R) se repiten sin variación; esto implica que varios bloques AES consecutivos son idénticos. Por ejemplo, una región de rojo puro (220, 50, 50) produce la secuencia de bytes [50, 50, 220, 50, 50, 220, ...] repetida, generando múltiples bloques de 16 bytes con contenido idéntico. ECB cifra cada bloque de forma independiente con la misma clave, por lo que bloques de plaintext idénticos producen ciphertext idéntico. En la imagen resultante, las regiones de color uniforme se transforman en regiones de un nuevo color uniforme (el ciphertext correspondiente), preservando la estructura de grilla del original. La imagen cifrada con ECB permite identificar visualmente la disposición de los bloques de colores originales.

Cifrado en modo CBC: el encadenamiento hace que cada bloque de 16 bytes se combine mediante XOR con el bloque ciphertext anterior antes de cifrarse. Aunque el plaintext de una región de color uniforme contiene bloques idénticos, sus ciphertexts son completamente distintos porque el “contexto” acumulado es diferente para cada posición. La imagen resultante tiene la apariencia de ruido pseudoaleatorio: ninguna estructura del original es discernible. Los tres archivos generados (original.bmp, ecb_encrypted.bmp, cbc_encrypted.bmp) tienen exactamente el mismo tamaño (49.206 bytes), ya que el encabezado BMP se preservó sin modificaciones y los datos de píxeles no requirieron padding.

Las Figuras 1, 2 y 3 ilustran los resultados del experimento:

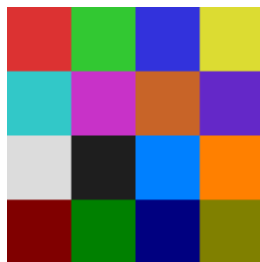


Fig. 1. Imagen original sintética.

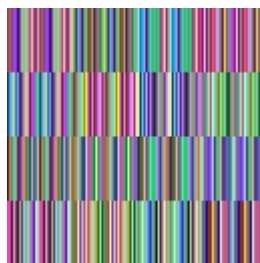


Fig. 2. Imagen cifrada con ECB. La estructura de grilla original es claramente visible.

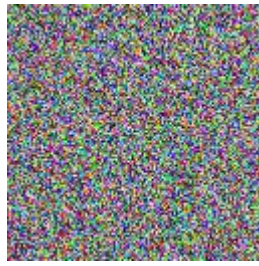


Fig. 3. Imagen cifrada con CBC. El encadenamiento elimina toda estructura reconocible.

C. Mejoras Futuras

Soporte para AES-192 y AES-256: la implementación actual cubre únicamente AES-128 (clave de 128 bits, 10 rondas). Extenderla a AES-192 (12 rondas) y AES-256 (14 rondas) requeriría ajustar el algoritmo de expansión de claves y el contador de rondas, sin cambios en las transformaciones de ronda.

Modos de operación adicionales: ECB y CBC son los modos más simples. En la práctica, el modo más utilizado actualmente es AES-GCM (Galois/Counter Mode), que combina cifrado (modo CTR) con autenticación de mensaje (GHASH), resolviendo además la vulnerabilidad de CBC ante ataques de padding oracle. Implementar CTR y GCM sería el paso natural para acercarse a un uso real.

Gestión de claves: la herramienta actual acepta claves en hexadecimal directo. En aplicaciones reales se usa una función de derivación de claves (KDF, como PBKDF2 o HKDF) para generar la clave AES a partir de una contraseña, incorporando un salt para evitar ataques de diccionario precalculados.

Rendimiento: al ser Python puro, la implementación es significativamente más lenta que una implementación en C o que las instrucciones AES-NI disponibles en procesadores modernos. Una extensión posible sería medir y comparar los tiempos contra la librería cryptography de Python (que usa AES-NI por debajo) para cuantificar la diferencia.

V. CONCLUSIONES

La implementación de AES-128 desde cero, sin librerías criptográficas externas, permitió verificar mediante los vectores oficiales del NIST que el algoritmo fue correctamente reproducido siguiendo el estándar FIPS-197. Los 18 tests ejecutados pasaron en su totalidad, incluyendo la coincidencia exacta con los valores intermedios del Apéndice B, un nivel de validación que solo es posible porque el estándar es completamente público y detallado.

El experimento visual con imágenes BMP confirmó empíricamente la debilidad estructural del modo ECB: al cifrar cada bloque de forma independiente, preserva la distribución estadística del mensaje original, lo que revela información sobre el contenido sin necesidad de conocer la clave. El modo CBC elimina esta vulnerabilidad mediante el encadenamiento, haciendo que cada bloque de ciphertext dependa de todos los bloques anteriores: el resultado es visualmente indistinguible del ruido aleatorio.

Desde el punto de vista matemático, el trabajo permitió comprender en profundidad la aritmética en $GF(2^8)$ y su rol central en MixColumns: la propiedad del cuerpo finito —que garantiza que la multiplicación de dos bytes siempre produce otro byte— es lo que hace posible que MixColumns sea una transformación invertible y eficiente, base de la difusión de AES. SubBytes, construida a partir de la inversa multiplicativa en $GF(2^8)$ compuesta con una transformación afín, es la única transformación no lineal del cifrador y la fuente de la confusión. Ambos conceptos fueron formulados por Shannon en 1949 [6] y AES constituye uno de los ejemplos más estudiados y rigurosos de su aplicación práctica.

La disponibilidad pública del estándar FIPS-197, con sus vectores de prueba detallados ronda por ronda, fue en este trabajo tanto una herramienta de validación como una demostración del principio de seguridad por diseño abierto: la confianza en AES no descansa en el secreto de su diseño, sino en décadas de escrutinio criptográfico sin que se haya encontrado ningún ataque práctico en condiciones normales de uso [8].

VI. REFERENCIAS Y BIBLIOGRAFÍA

A. Referencias bibliográficas

- [1] National Bureau of Standards, “Data Encryption Standard,” FIPS PUB 46, Jan. 1977.
- [2] Electronic Frontier Foundation, “Cracking DES: Secrets of Encryption Research, Wiretap Politics & Chip Design,” O’Reilly Media, 1998.
- [3] NIST, “Announcing Request for Candidate Algorithm Nominations for the Advanced Encryption Standard (AES),” Federal Register, vol. 62, no. 177, Sep. 1997.
- [4] J. Daemen and V. Rijmen, “AES Proposal: Rijndael, Version 2,” NIST AES submission, Sep. 1999.
- [5] NIST, “Advanced Encryption Standard (AES),” FIPS PUB 197, Nov. 2001.
- [6] C. E. Shannon, “Communication Theory of Secrecy Systems,” Bell System Technical Journal, vol. 28, no. 4, pp. 656–715, Oct. 1949.
- [7] Committee on National Security Systems, “CNSS Policy No. 15, Fact Sheet No. 1: National Policy on the Use of the Advanced Encryption Standard (AES),” Jun. 2003.
- [8] A. Biryukov and D. Khovratovich, “Related-key Cryptanalysis of the Full AES-192 and AES-256,” in Proc. ASIACRYPT 2009, Lecture Notes in Computer Science, vol. 5912, pp. 1–18, Springer, 2009.

B. Bibliografía

- W. Stallings, *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson, 2017.
- J. Daemen & V. Rijmen, *The Design of Rijndael: AES — The Advanced Encryption Standard*. Springer-Verlag, 2002.
- N. Ferguson, B. Schneier, & T. Kohno, *Cryptography Engineering: Design Principles and Practical Applications*. Wiley, 2010.
- C. Paar & J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer, 2010.